Phase 2A Dynamic FJSP Minimal Model

FJSP-AGV-RL Research

June 2026

1 Scope

Phase 2A extends the verified Phase 1 static flexible job shop scheduling platform with a minimal dynamic layer. The current scope is deliberately small: dynamic events, a dynamic state, frozen records, remaining-operation views, and deterministic rescheduling. AGV routing, transportation time, reinforcement learning, graph neural networks, operation preemption, and multi-objective optimization remain outside this phase.

2 Static Base

Let J be the set of jobs and let job $j \in J$ contain ordered operations $O_j = \{0, \ldots, n_j - 1\}$. Each operation (j, o) has an eligible machine set M_{jo} and processing time p_{jom} for each $m \in M_{jo}$. A schedule record is

$$r = (j, o, m, s, e, p),$$

where s is start time, $e = s+p$ is end time, and $p = p_{jom}$. Phase 2A keeps the same `ScheduleRecord` and `ScheduleResult` representation used by Phase 1.

3 Dynamic State

At rescheduling time t, the dynamic state is

$$S_t = (I_t, t, R_t, a_t, u_t),$$

where:

- I_t is the current FJSP instance.
- $R_t \subseteq J_t$ is the set of released jobs.
- $a_t(j)$ is the release time of job j.
- $u_t(m)$ is the earliest time machine m can start a new future operation.

The implementation stores a_t as `job_release_times` and u_t as `machine_unavailable_until`.

4 Events

The Phase 2A event set is shown in Table 1. Events are applied in non-decreasing time order by `apply_events()`.

Event	State effect
`MachineBreakdown`	Set $u_t(m)$ to at least the repair time.
`MachineRepair`	Set $u_t(m)$ to the repair event time.
`JobArrival`	Append a new job and set its release time.
`ProcessingTimeChange`	Update one future p_{jom}.

Table 1: Phase 2A dynamic event types.

5 Frozen Records

Given an existing schedule x and current time t, Phase 2A defines the frozen record set as

$$F_t = \{r \in x : e_r \le t\} \cup \{r \in x : s_r < t < e_r\}.$$

The second set freezes in-process operations because Phase 2A does not model preemption. The helper `frozen_records_at_time()` implements this rule.

Frozen records must be copied exactly into the rescheduled result:

$$(m'_r, s'_r, e'_r, p'_r) = (m_r, s_r, e_r, p_r), \quad r \in F_t.$$

6 Remaining Operations

For each released job j, frozen operations must form a route prefix:

$$F_{tj} = \{0, \ldots, k_j - 1\}.$$

The remaining operations are then

$$Q_t = \{(j, o) : j \in R_t,\ o \in \{k_j, \ldots, n_j - 1\}\}.$$

The helper `remaining_operations_from_frozen()` exposes Q_t for audit and later repair policies.

7 Rescheduling Baseline

The current baseline dispatches all operations in Q_t while respecting:

$$s_{jo} \ge a_t(j), \qquad \text{job release time,} \tag{1}$$

$$s_{jo} \ge u_t(m), \qquad \text{machine availability for future starts,} \tag{2}$$

$$s_{j,o+1} \ge e_{jo}, \qquad \text{job precedence,} \tag{3}$$

$$[s_{jo}, e_{jo}) \cap [s_{j'o'}, e_{j'o'}) = \emptyset, \qquad \text{same-machine non-overlap.} \tag{4}$$

The implementation currently supports deterministic `fifo`, `spt`, and `eft` rules. The returned result is still a full `ScheduleResult`; it is checked by both the Phase 1 static feasibility checker and the Phase 2A dynamic feasibility checker.

8 Smoke Verification

Experiment `exp_014_dynamic_reschedule_smoke.py` currently validates three small scenarios:

Scenario	Makespan	Feasible
`dynamic_breakdown_1x1`	12	PASS
`dynamic_job_arrival_2x1`	10	PASS
`dynamic_multi_event_2x2`	8	PASS

Table 2: Current Phase 2A dynamic smoke results.

9 Limitations

The current model is intentionally conservative. It does not interrupt running operations, add transport resources, optimize AGV paths, simulate stochastic event streams, or train policies. These restrictions keep Phase 2A small enough to audit while preserving the Phase 1 legality guarantees.